HW5 Worksheet

Deriving classes and functions from one system requirement

Name: Alvin Ramcharran Date: 10/13/2026

Purpose: Apply the same method used in lecture: SR → behavior → responsibility → needed information → owner → function → coordination → AT. Choose one requirement and show evidence for every design decision.

Choose one requirement

Choice	Requirement focus
SR-05	**status classifies Working, Stage, and HEAD observations**
SR-08	**commit creates a complete snapshot and updates HEAD safely**

Selected requirement: SR-05

Shared project model

Working Tree contains ordinary project files. Stage contains bytes selected by add. HEAD identifies the latest committed snapshot. .minigit is the MiniGit-owned metadata boundary. Assume C++17, project-relative paths, exact byte comparison, and Error: ... with exit status 1 on failure.

Step 1: Observable behavior

Write concrete Before, Trigger, After, and Must-not-change statements. Each answer must be observable through output, exit status, or filesystem bytes.

Field	Your answer
Before	The project is initialized. The HEAD commit holds notes.txt = ONE and old.txt = OLD. notes.txt was staged as TWO, then the working copy was edited to THREE. plan.txt = PLAN was never added. old.txt was deleted from the Working Tree.
Trigger	minigit status
After	Exit status 0. stdout lists notes.txt as staged and modified after staged, plan.txt as untracked, and old.txt as deleted.
Must-not-change	Every Working file keeps its bytes (notes.txt THREE, plan.txt PLAN), and old.txt is not recreated. The staged notes.txt stays TWO. The index, HEAD, and commits under .minigit are byte-for-byte unchanged.

Step 2: Responsibilities

Write responsibilities as verbs. Avoid vague phrases such as “handle status” or “manage commit.”

Responsibility	What work must the system perform?
R1	Confirm the project is initialized before reading any records.
R2	Collect every path to report from the Working Tree, the index, and the HEAD snapshot, listing each path once.
R3	Observe each path by checking where it exists and comparing bytes between each pair of versions that exist.
Add if needed	R4. Classify each path into one or more labels from its observations.
	R5. Print each path with its labels and exit 0, or print Error: ... and exit 1.

Step 3: Needed information

For each responsibility, list data, rules, current state, and error policy. Include boundary information such as missing versus empty, protected paths, duplicate paths, parent IDs, or the point at which HEAD may change.

Responsibility	Needed information	Why this information is needed
R1	Whether .minigit and HEAD exist. A HEAD that names no commit still counts as initialized. If HEAD is missing, status prints Error: ... and exits 1.	A new project with no commits must still report untracked files. A folder that was never initialized must fail instead of guessing.
R2	The Working Tree file list with .minigit skipped. The tracked paths in the index. The file list of the HEAD commit, which is empty when HEAD names no commit. Each path appears once even when it is in all three.	Untracked files only show up in the Working Tree, and deleted files only show up in the index or HEAD. Skipping .minigit keeps MiniGit's own records out of the report (SR-10).
R3	For each path, which of Working, Stage, and HEAD hold it, and their exact bytes. A zero-byte file is present, which is different from a missing file. An equality only counts when both versions exist. A file that cannot be read stops the command with Error: ... and exit 1.	The six labels depend on presence and on exact byte equality. Treating missing as empty would hide deleted files.
Add if needed	R4. The six label rules from Lab 6 section 3B. A path can match more than one rule. The observations are kept as separate flags, not one stored state.	A file can be staged and also modified after staged at the same time, so one stored state would lose information.
	R5. The output format, a fixed path order, and exit codes 0 and 1.	Tests compare stdout, so the same project must always print the same

Responsibility	Needed information	Why this information is needed
		lines in the same order.

Step 4 Owner: Information Expert and Controller

Information Expert owns the information needed for a focused responsibility. The controller receives the system operation and coordinates the order.

Responsibility	Information Expert owner	Controller or coordinator	SR evidence
R1 Confirm initialized	RepositoryPaths	MiniGitRepository::status()	SR-05A to SR-05F, shared project model
R2 Collect paths	FileSystem (Working list), IndexTable (staged paths), Snapshot (HEAD paths)	MiniGitRepository::status()	SR-05A, SR-05E, SR-10
R3 Observe each path	FileSystem and Snapshot (bytes), FileState (holds the flags)	MiniGitRepository::status()	SR-05B to SR-05F
R4 Classify	FileState, through classify()	MiniGitRepository::status()	SR-05A to SR-05F
R5 Print and exit	main (program boundary)	MiniGitRepository::status()	SR-05A to SR-05F, shared project model

If you propose a new class, explain why an existing class cannot own the responsibility without losing cohesion.

Snapshot and FileState are new. Snapshot owns the file list and bytes of one commit. IndexTable only knows staged paths and HeadReference only knows which commit is current, so neither can answer what HEAD contained without taking on a second job. FileState holds one path's observations, and classify() turns them into labels. Putting the six label rules in MiniGitRepository would make the Controller own rules instead of coordinating. In FileState they stay in one place that can be tested without touching the disk.

Step 5: Candidate functions and contracts

Include inputs, output or effect, precondition, failure behavior, and the SR source for each function.

Owner	Function	Inputs and output/effect	Preconditions and failure behavior	SR source
RepositoryPaths	requireInitialized()	No input. Returns nothing when .minigit and HEAD exist.	Throws "not a MiniGit project" when HEAD is missing. Reads only.	SR-05A to SR-05F
FileSystem	listFiles(const fs::path& root)	Project root in. Returns project-relative paths of regular files, with .minigit skipped.	Root must be a directory. Throws if a folder cannot be read.	SR-05A, SR-10

Owner	Function	Inputs and output/effect	Preconditions and failure behavior	SR source
IndexTable	paths()	No input. Returns the staged paths. An empty index gives an empty list.	A missing index throws, because an initialized project always has one.	SR-05B, SR-05C
HeadReference	current()	No input. Returns the current commit ID, or none when no commit exists yet.	HEAD must exist. Throws if HEAD cannot be read.	SR-05D to SR-05F
Snapshot	paths() and bytes(path)	Commit ID in. Returns that commit's file list, or one file's bytes.	With no commit, paths() returns an empty list. Throws if the commit's records are missing.	SR-05D to SR-05F
FileSystem	readBytes(const fs::path&)	Path in. Returns the file's exact bytes.	The file must exist. Throws if it cannot be opened.	SR-05B to SR-05F
FileState	classify(const FileState&)	Six observation flags in. Returns one or more labels.	No precondition and no failure. An equality flag only counts when both versions exist.	SR-05A to SR-05F
MiniGitRepository	status()	No input. Returns every path with its labels in path order.	Writes nothing. Any error stops it before anything is printed.	SR-05A to SR-05F
main	main(argc, argv)	Reads "status". Prints the results and returns 0.	Catches errors, prints Error: ..., and returns 1.	Shared project model

Step 6: Coordination sequence

Write the call order. Mark the first mutation and the publication point.

Order	Operation
1	RepositoryPaths::requireInitialized() checks for .minigit and HEAD.
2	HeadReference::current() reads the current commit ID, which may be none.
3	FileSystem::listFiles(), IndexTable::paths(), and Snapshot::paths() collect the paths, each listed once.
4	For each path, FileSystem::readBytes() and Snapshot::bytes() read the versions that exist.
5	Build a FileState for each path from presence and exact byte comparisons.
6	classify() turns each FileState into its labels.
7	main prints every path with its labels in path order and returns 0. PUBLICATION POINT. Nothing is

Order	Operation
	printed until every path is classified. NO MUTATION at any step, because status is read-only.

Which object controls the order? MiniGitRepository, through status().

Which operation must happen before any mutation? status makes no mutation. RepositoryPaths::requireInitialized() runs first, before any record is read.

What valid state must remain if a later operation fails? Everything stays as it was, because status writes nothing. Only Error: ... appears on stderr with exit 1, and no partial list is printed.

Step 7 Acceptance-test traceability

Design at least five observations. At least one must be negative or a boundary case.

Observation	Expected result	Responsible class/function	Step verified
status output for notes.txt in the Before state	Listed as staged and modified after staged	FileState, classify()	Step 6, orders 4 to 6
status output for plan.txt	Listed as untracked	FileSystem::listFiles(), classify()	Step 6, orders 3 and 6
status output for old.txt, and the folder after status	Listed as deleted, and old.txt is still absent	Snapshot::paths(), classify()	Step 6, orders 3 and 6
Commit notes.txt = ONE, edit it to TWO without add, run status	Listed as modified only. A second test with no edit lists it as clean	classify()	Step 6, order 6
Bytes of every Working file, the Stage, the index, and HEAD before and after status	All identical	MiniGitRepository::status()	Step 6, every order (no mutation)
Boundary: initialized project with no commits, an empty index, and plan.txt	Exit 0, plan.txt listed as untracked	HeadReference::current(), IndexTable::paths()	Step 6, orders 2 and 3
Negative: run status in a folder with no .minigit	Error: ... on stderr, exit 1, nothing on stdout	RepositoryPaths::requireInitialized()	Step 6, order 1